School of Future Tech

Case Study Report

on

Scholarship Management System

(Case Study 61)

by

Harsh Kumar

Roll NO. 150096725105

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

2. Problem Statement / Case Background (Abstract)

c rou

Universities manage multiple scholarship programmes with different eligibility criteria. Manually reviewing each student's GPA and family income, cross-checking against multiple scholarship types, and generating status reports is time-consuming and susceptible to human error.

tr ct

This case study presents the design and implementation of a Scholarship Management System in C++. The system maintains a set of scholarship definitions (with GPA thresholds and income limits) and student records. It automatically checks each student's eligibility against all available scholarships, assigns the appropriate scholarship or marks the student as rejected, and displays a formatted status report on the console.

The implementation uses two primary classes — Scholarship and Student — managed by a ScholarshipSystem controller class. Input validation is handled through dedicated helper functions that use getline-based reading to ensure robust, clean user input. The system was designed and built as part of the B.Tech CSE Semester II C++ coursework at ITM Skills University.

3. Case Study Design

The system is designed around three core classes that represent real-world entities, managed by a central system class. The design follows encapsulation principles — all data members are private and accessed through defined getter and setter methods.

Scholarship Class — stores scholarship name, minimum GPA, maximum family income, and fund amount. Includes a checkEligibility() method returning a descriptive reason string if a student is ineligible.
Student Class — stores student ID, name, GPA, family income, scholarship status (defaulting to 'Pending'), and allocated funds. Includes a displayStatus() method that prints a formatted bordered card to the console.
ScholarshipSystem Class — the central controller. Manages arrays of up to 100 students and 10 scholarships, provides methods for adding students and scholarships, processing eligibility, displaying reports, and tracking individual students by ID.

S te o :
Program starts and pre-loads two scholarships: Merit Base (GPA >= 8.5, Income <= 8,00,000, Rs. 50,000) and Need Base (GPA >= 7.0, Income <= 4,00,000, Rs. 30,000).

User interacts through a numbered menu: Add Student, Process Scholarships, View Report, Track by ID, Exit.

After adding students, the user runs Process Scholarships. The system iterates over all students and for each checks all scholarships in order, assigning the first matching one.

If no scholarship matches, the student is marked as Rejected.

The report displays all students in a bordered card format with ID, Name, GPA, Status, and Funds.

rc tecture:

Class	Responsibility	Key Members
Scholarship	Stores scholarship criteria and checks eligibility	name, minGPA, maxIncome, fundAmount, checkEligibility()
Student	Stores applicant data and tracks status	studentID, name, gpa, familyIncome, scholarshipStatus, allocatedFunds
ScholarshipSystem	Central manager — handles all operations	addStudent(), processScholarships(), trackStudentByID()

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

t er c t o or t :

The core eligibility check iterates through all available scholarships for each student. For each scholarship, the student's GPA is compared against the minimum required GPA, and their family income is compared against the maximum allowed income. The first scholarship whose criteria the student meets is assigned to them. If no scholarship matches, the system calls checkEligibility() on the first scholarship to return a specific rejection reason rather than a generic message.

u c te etect o :

Before adding a new student, the system iterates through the existing students array. If any existing student shares the same ID, the addition is blocked with an error message — preventing data corruption.

ut t o :

Three overloaded helper functions (getValidID(), getValidGPA(), getValidIncome()) enforce input constraints:
Student ID must be a positive integer
GPA must be a decimal value between 0.0 and 10.0
Family income must be a non-negative number
Each function loops until valid input is received, using cin.clear() and cin.ignore() to recover from bad input states.

ec o o St c :

- Language: C++ (standard C++11 or later)
- Libraries: <iostream> and <string> only — no file I/O, no external dependencies
- Compiler: g++ (tested on macOS and Linux)
- IDE: VS Code with C++ extension
- Data storage: In-memory arrays (no persistence between runs)

P o ce t e o tr te :

- Encapsulation — data members and methods grouped within classes
- Default Constructors — all classes initialise members to safe defaults
- Method Design — each class exposes only the behaviour it owns
- Separation of Concerns — input helpers are standalone functions, not mixed into class logic

5. Problem Statement / Case Study Implementation Details and Snapshots.

Project Structure :

The entire system is contained in a single .cpp file organised into clearly commented sections: class definitions, system controller, input helpers, and main menu loop.

```cpp
// Class defining the Scholarship criteria
class Scholarship {
public:
    string name;
    double minGPA;
    double maxIncome;
    double fundAmount;

    // Default Constructor
    Scholarship() {
        name       = "";
        minGPA     = 0;
        maxIncome  = 0;
        fundAmount = 0;
    }
```

```cpp
// Class defining the Student
class Student {
public:
    int    studentID;
    string name;
    double gpa;
    double familyIncome;
    string scholarshipStatus;
    double allocatedFunds;

    // Default Constructor
    Student() {
        studentID         = 0;
        name              = "";
        gpa               = 0;
        familyIncome      = 0;
        scholarshipStatus = "Pending";
        allocatedFunds    = 0.0;
    }
```

```cpp
void displayStatus() {
    cout << "  +-----------------------------------------------+\n";
    cout << "  | ID        : " << studentID                      << "\n";
    cout << "  | Name      : " << name                           << "\n";
    cout << "  | GPA       : " << gpa                            << "\n";
    cout << "  | Status    : " << scholarshipStatus              << "\n";
    cout << "  | Funds     : Rs. " << allocatedFunds             << "\n";
    cout << "  +-----------------------------------------------+\n";
}
```

```cpp
string checkEligibility(double gpa, double income) {
    if (gpa < minGPA && income > maxIncome)
        return "GPA too low & Family income exceeds limit";
    if (gpa < minGPA)
        return "GPA too low";
    if (income > maxIncome)
        return "Family income exceeds limit";
    return "ELIGIBLE";
}
```

```cpp
// Input Validation Helpers
int getValidID() {
    int id;
    string line;
    while (true) {
        getline(cin, line);
        bool valid = true;
        if (line.empty()) valid = false;
        for (char c : line) if (!isdigit(c)) { valid = false; break; }
        if (valid) {
            id = stoi(line);
            if (id > 0) return id;
        }
        cout << "\nInvalid ID. Enter a positive integer: " << endl;
    }
}

double getValidGPA() {
    double gpa;
    string line;
    while (true) {
        getline(cin, line);
        try {
            size_t pos;
            gpa = stod(line, &pos);
            if (pos == line.size() && gpa >= 0.0 && gpa <= 10.0) return gpa;
        } catch (...) {}
        cout << "\nInvalid GPA. Enter a value between 0.0 and 10.0: " << endl;
    }
}

double getValidIncome() {
    double income;
    string line;
    while (true) {
        getline(cin, line);
        try {
            size_t pos;
            income = stod(line, &pos);
            if (pos == line.size() && income >= 0) return income;
        } catch (...) {}
        cout << "\nInvalid income. Enter a positive value: Rs. " << endl;
    }
}
```

```cpp
switch (choice) {
    case 1: {
        cout << "\n--- Enter Student Details ---\n";
        cout << "Student ID: ";
        int id = getValidID();

        string name;
        cout << "Full Name: ";
        getline(cin, name);

        cout << "GPA (0.0 - 10.0): ";
        double gpa = getValidGPA();

        cout << "Family Income: Rs. ";
        double income = getValidIncome();

        Student newStudent;
        newStudent.studentID    = id;
        newStudent.name         = name;
        newStudent.gpa          = gpa;
        newStudent.familyIncome = income;

        sys.addStudent(newStudent);
        break;
    }
    case 2: sys.processScholarships(); break;
    case 3: sys.displayReport();       break;
    case 4: {
        cout << "\nEnter Student ID to track: ";
        int id = getValidID();
        sys.trackStudentByID(id);
        break;
    }
    case 5:
        cout << "\nExiting Scholarship Management System. Goodbye!\n";
        running = false;
        break;
    default:
        cout << "\nInvalid choice. Please enter a number between 1 and 5.\n";
}
```

✓ Output

```
==========================================
  SCHOLARSHIP MANAGEMENT SYSTEM MENU
==========================================
1. Add New Student
2. Process Scholarships (Verify Eligibility)
3. View Full Status Report (Console)
4. Track a Specific Student by ID
5. Exit System
Enter your choice (1-5): 1

--- Enter Student Details ---
Student ID: 101
Full Name: Harsh Kumar
GPA (0.0 - 10.0): 9.0
Family Income: Rs. 500000

--- Student 'Harsh Kumar' added successfully! ---

==========================================
  SCHOLARSHIP MANAGEMENT SYSTEM MENU
==========================================
1. Add New Student
2. Process Scholarships (Verify Eligibility)
3. View Full Status Report (Console)
4. Track a Specific Student by ID
5. Exit System
Enter your choice (1-5): 1

--- Enter Student Details ---
Student ID: 102
Full Name: Ayush Kumar Singh
GPA (0.0 - 10.0): 9.5
Family Income: Rs. 400000

--- Student 'Ayush Kumar Singh' added successfully! ---
```

```
=====================================
  SCHOLARSHIP MANAGEMENT SYSTEM MENU
=====================================
1. Add New Student
2. Process Scholarships (Verify Eligibility)
3. View Full Status Report (Console)
4. Track a Specific Student by ID
5. Exit System
Enter your choice (1-5): 2

  ===============================================
       Scholarship Processing Complete!
  ===============================================
```

```
=====================================
  SCHOLARSHIP MANAGEMENT SYSTEM MENU
=====================================
1. Add New Student
2. Process Scholarships (Verify Eligibility)
3. View Full Status Report (Console)
4. Track a Specific Student by ID
5. Exit System
Enter your choice (1-5): 3

  ==============================================
      ITM SKILLS UNIVERSITY
      Scholarship Status Report
  ==============================================
  Total Students: 2

  +-----------------------------------------------+
  | ID       : 101
  | Name     : Harsh Kumar
  | GPA      : 9
  | Status   : Approved - Merit Base
  | Funds    : Rs. 50000
  +-----------------------------------------------+
  +-----------------------------------------------+
  | ID       : 102
  | Name     : Ayush Kumar Singh
  | GPA      : 9.5
  | Status   : Approved - Merit Base
  | Funds    : Rs. 50000
  +-----------------------------------------------+
  ==============================================
```

6. Problem Statement / Case Study Results and Conclusion.

e u t :

The system successfully demonstrates all three functional requirements from the problem statement:

- **Student Eligibility Verification** — the processScholarships() method correctly evaluates all students against all loaded scholarships using GPA and income conditions.
- **Scholarship Allocation** — eligible students are assigned the first matching scholarship with the correct fund amount. No double-allocation occurs due to the break-on-match logic.
- **Status Tracking** — both the full report (displayReport) and individual lookup (trackStudentByID) work correctly, displaying clean formatted output.

Input validation was strengthened throughout development: ID, GPA, and income validators use getline-based reading to prevent buffer leftover issues; the name validator uses ASCII range checks to reject numeric or special character input without requiring extra libraries.

Functional Requirement	Implementation	Outcome
Student eligibility verification	checkEligibility() in Scholarship class	Specific rejection reason shown to user
Scholarship allocation	processScholarships() in ScholarshipSystem	Best match assigned; fund amount recorded
Status tracking	trackStudentByID() with ID lookup	Instant status retrieval by student ID

o c u o :

This case study demonstrates a complete, self-contained scholarship management application built entirely with core C++ constructs. It integrates OOP class design, input validation, eligibility logic, and console-based reporting into a coherent, working system.
The project meets all deliverables specified in the problem statement: a functional scholarship system with eligibility verification, allocation logic, status tracking, and formatted console reports. It provides a strong foundation for future enhancements such as file persistence, sorting, or a GUI interface.

7. References

- Bjarne Stroustrup — The C++ Programming Language (4th Edition), Addison-Wesley
- cppreference.com — C++ Standard Library Reference (iostream, string, etc.)
  https://en.cppreference.com/w/cpp/string/basic_string/getline
- ITM Skills University C++ Syllabus — B.Tech CSE Semester II Course Material
- GeeksForGeeks — OOP in C++: Classes, Objects, Constructors, Encapsulation (online)
  https://www.geeksforgeeks.org/c-classes-and-objects/
- VS Code C++ Extension Documentation — for compiler setup and debugging environment configuration.